STRANGE BOOK / OPENLAB

奇怪的书

三道任务 · 全解

Ag 在校园里捡到一本写满奇怪符号的书。符号只有八种，
可是它们合在一起，居然能算出字符、打印文字、做加法。
这份解答把书末留下的三道任务逐条拆开，
连同每一步的指针位置与单元格状态一起写清楚。

任务	位置	考察点	结论
任务一	第一页	循环搬运、输出减一	输出输入字符的前一个字符
任务二	第二页	循环凑常量、双格输出	输出“OP”（79, 80）
任务三	第三页	读入两字符、加法合并	输出两字符 ASCII 值之和

语言速查：八种关键字

符号	含义	符号	含义
+	指针所指单元值 +1	[	值为 0 时，跳到向后最近的]
−	指针所指单元值 −1	]	值非 0 时，跳回向前最近的 [
>	指针向右移动一格	.	输出该单元的 ASCII 字符
<	指针向左移动一格	,	读入一个字符存到该单元

单元格值域 0–255，越界即回绕（255 + 1 = 0，0 − 1 = 255）。指针初始位于第 0 格，所有格子初值均为 0。



指针 初始指向第 0 格，格子向左右无限延伸

TASK 01 • 第一页

Yimer 的考题：这段程序写了啥?

书页上只有五行符号，中间夹着两个换行。换行不是关键字，真正的程序只有 ,[>+<-]>-. 这九个符号。

```
,[
>+
<-
]
>-.
```

逐段拆解

① , 读入一个字符，写进第 0 格，设它的 ASCII 值为 c。

② […] 第 0 格为 0 就整体跳过；否则进入循环。循环体是 +（第 1 格加 1）和 -（第 0 格减 1），等于把 c 从第 0 格一格一格搬到第 1 格，来回搬 c 次。

③ > - . 指针移到第 1 格（此时为 c），减 1 后作为 ASCII 打印。

> **程序做的事**
> 读一个字符 → 搬到下一格 → 减 1 → 打印。
> 换句话说：输出**输入字符在 ASCII 表里的前一个字符**（ASCII 值减 1）。

跑一遍：输入 A（65）

阶段	第 0 格	第 1 格	说明
初始	0	0	指针在第 0 格
读入 A	65	0	c = 65
第 1 轮循环后	64	1	搬走一个数，指针回到第 0 格
第 65 轮循环后	0	65	全部搬完，循环条件不成立，退出
>-	0	64	第 1 格减 1
.	0	64	输出 64 → '@'

几个常见输入

输入	ASCII	输出	ASCII	输入	ASCII	输出	ASCII
A	65	@	64	a	97	`	96
Z	90	Y	89	0	48	/	47
!	33	空格	32	~	126	}	125

答案（输入 → 输出）

例：{A(65) → @(64)}，{Z(90) → Y(89)}，{!(33) → 空格(32)}。

一句话：**输出紧挨在输入字符前面的那个字符**，相当于查完 ASCII 表把值减去 1。边界上，若输入 NUL（0），循环一次也不执行，第 1 格仍是 0，减 1 按回绕变成 255，会打印一个 0xFF 字节。

TASK 02 • 第二页

Yimer 的困惑：不用输入，它打印了什么?

这一页里没有读入指令，不需要任何输入，输出完全由符号本身决定——这就是“常量打印程序”。

```
++++++++++
[
>++++++++
>++++++++
<<-
]
>-.
>.
```

逐段拆解

① `++++++++++` 先把第 0 格写成 10，它是外层循环的计数器。

② `[ … ]` 每轮：右移给第 1 格加 8，再右移给第 2 格加 8，然后 `<<-` 回到第 0 格把计数器减 1；计数器不为 0 就再来一轮。

③ 一共转 10 轮，第 1 格和第 2 格都等于 10 × 8 = 80。

④ `>-.` 第 1 格 80 − 1 = 79，输出 'O'。

⑤ `>.` 第 2 格仍是 80，输出 'P'。

> **程序做的事**
> 不读输入，直接算出两个常量并依次打印：先是 79，再是 80。
> 屏幕上最终出现两个字符：**O** 和 **P**。

循环过程

轮次	第 0 格（计数）	第 1 格	第 2 格	本轮结束
0	10	0	0	—
1	9	8	8	继续下一轮
2	8	16	16	继续下一轮
5	5	40	40	继续下一轮
10	0	80	80	计数为 0，退出循环

轮次 3、4 与 6–9 依此类推：每转一轮，两个目标格各加 8。

答案

输出：**OP**，即 {O(79), P(80)}。

为什么写 `10 × 8` 而不是直接敲 80 个 +？因为外层循环把“重复”交给计数器去做，代码更短、也不容易数错——这正是用循环“凑常量”的常见写法。

TASK 03 • 第三页

这一页要自己写：两字符 ASCII 值求和

题目：读入两个字符，把它们的 ASCII 值相加，输出结果对应的字符。思路很直接——读进来、加起来、打出去。

参考程序

```
,>,[<+>-]<.
```

共 11 个符号。

逐段拆解

① `,` 读第一个字符，存进第 0 格（设为 a）。

② `>` `,` 指针右移，读第二个字符，存进第 1 格（设为 b）。

③ `[<+>-]` 经典的“搬运加法”：每轮把第 1 格减 1、第 0 格加 1，直到第 1 格归零，此时第 0 格就是 a + b，第 1 格被清空。

④ `<` `.` 指针回到第 0 格，输出这个和对应的字符。

> **等价的“人类语言”版本**
>
> `a = read()  b = read()  print(chr(a + b))`
>
> 在值域 0–255 上取模，因此两数之和不超过 255 时就是普通的加法。

运行示例：输入 “23”

阶段	第 0 格	第 1 格	输出	说明
初始	0	0	—	指针在第 0 格
读 '2'	50	0	—	a = 50
`>` `,` 读 '3'	50	51	—	b = 51
循环结束	101	0	—	50 + 51 = 101，第 1 格清零
`<` `.`	101	0	e	101 对应字符 'e'

更多例子

输入	ASCII 值	和	输出字符
“23”	50 + 51	101	e (101)
“11”	49 + 49	98	b (98)
“A ”	65 + 32	97	a (97)
“!!”	33 + 33	66	B (66)

答案

程序：`,>,[<+>-]<.` 即读入两个字符、把它们的 ASCII 值相加、输出和所对应的字符。

注意值域：格子只能存 0–255，和一旦超过 255 就会回绕（如 200 + 100 = 44）。标准可打印 ASCII 最大 126，两数相加最多 252，不会溢出；若允许输入 ≥128 的扩展字符，就要接受回绕结果。

APPENDIX • 验证

用一个解释器复核所有结论

上面每一句结论都不是“看着像”，而是把三段程序丢进一个 Python 写的解释器里跑出来的：值域按 256 取模、括号跳转按“最近匹配”实现，与书上规则完全一致。

```
def run(code, inp):
 code = ''.join(c for c in code if c in '+-<>[].,')
 m, st = {}, [] # bracket pairs
 for i, c in enumerate(code):
 if c == '[': st.append(i)
 elif c == ']':
 j = st.pop(); m[i], m[j] = j, i
 tape, p, ip, iq, out = [0] * 3000, 0, 0, 0, []
 while ip < len(code):
 c = code[ip]
 if c == '+': tape[p] = (tape[p] + 1) % 256
 elif c == '-': tape[p] = (tape[p] - 1) % 256
 elif c == '>': p += 1
 elif c == '<': p -= 1
 elif c == '.': out.append(tape[p])
 elif c == ',':
 tape[p] = ord(inp[iq]) if iq < len(inp) else 0; iq += 1
 elif c == '[' and tape[p] == 0: ip = m[ip]
 elif c == ']' and tape[p] != 0: ip = m[ip]
 ip += 1
 return out
```

复核输出

程序	输入	输出值	输出字符	结论
第一页	A	64	@	65 − 1 = 64 √
第一页	Z	89	Y	90 − 1 = 89 √
第一页	!	32	空格	33 − 1 = 32 √
第二页	（无）	79, 80	OP	10×8−1 与 10×8 √
第三页	23	101	e	50 + 51 = 101 √
第三页	11	98	b	49 + 49 = 98 √

结语

八个符号、一条纸带、一个指针，就够做加法和打印了。第一页教你“搬运”，第二页教你“用循环凑常量”，第三页要你把两者接起来——所谓图灵完备，大概就是从这三件小事开始的。

（完）